

Capítulo 1

- Problemas: O acúmulo de código confuso reduz a produtividade da equipe, gerando atrasos, bugs incorrigíveis e ao replanejamento do projeto.
 - O código é a linguagem na qual expressamos nossos requisitos.
 - Um código ruim é como um lamaçal de arbustos emaranhados com armadilhas ocultas.
 - A pressa é inimiga da perfeição.
 - Um programa confuso que funcione pode ser pior que nada.
 - Uma alteração no código causa uma falha em outras duas ou três partes do mesmo código, exigindo restaurações, amarrações e remendos.
 - A necessidade de reescrever o projeto se torna obrigatória, aumentando os gastos do projeto, o tempo perdido para o replanejamento do sistema, e uma nova equipe para a manutenção do sistema atual e do novo projeto.
 - Falta de profissionalismo, ceder a vontade dos gerentes que não entendem os riscos de gerar códigos confusos.

- Ideia central: O código limpo não é apenas um detalhe técnico, mas uma questão de sobrevivência profissional e ética; escrever código que funciona não é suficiente, ele deve ser mantido limpo para permitir a evolução do sistema.
 - Você não cumprirá o prazo se fizer bagunça no código, tal desorganização reduzirá instantaneamente a velocidade de trabalho e você perderá o prazo. A única maneira de isso não acontecer – a única maneira de ir mais rápido - é sempre manter o código limpo.
 - Escrever código limpo exige o uso disciplinado de pequenas técnicas aplicadas por meio de uma sensibilidade adquirida sobre ‘limpeza’, a ‘sensibilidade ao código’ é o segredo.

- Objetivo: Definir o conceito de “código limpo” através da visão de especialistas e estabelecer que o programador é o único responsável pela clareza do código.
 - Não existe bala de prata, ou seja, não existe uma técnica única que resolva todos os problemas, o programador deve expandir seus conhecimentos e práticas (a prática faz o mestre).
 - Existem muitas recomendações sobre código limpo que são contraditórias e que podemos discordar, porém é obrigação do desenvolvedor analisar cada problema e solução para encontrar o melhor caminho para o código limpo.

- Considere o fato de que a taxa de tempo na leitura vs escrita é de 10x1, passamos mais tempo lendo um código antigo do que escrevendo um código novo, por isso a leitura do código deve ser fácil
- Conceitos essenciais: Nunca deixa para amanhã o que pode fazer hoje, a lei de LeBlanc destaca a importância de limpar um código sempre que se vê uma oportunidade, a sensibilidade ao código é fundamental para deixar o código pelo menos um pouco mais limpo do que você o encontrou antes.
 - Regra de escoteiro: Um código precisa ser mantido sempre limpo para que ele não se degrade com o tempo (Deixe a área do acampamento mais limpa do que como você encontrou). A limpeza não precisa ser algo grande, trocar o nome de uma variável, dividir uma função que esteja um pouco grande, eliminar um pouco de repetição, reduzir uma instrução if aninhada são atitudes que simplesmente melhora o código com o tempo, o aperfeiçoamento continuo te faz um profissional melhor.
 - Lei de LeBlanc: “Mais tarde significa nunca”, quando melhorias necessárias são continuamente adiadas ao invés de serem realizadas no momento adequado pode causar problemas no sistema sendo desenvolvido, porém isso não deve ser interpretado de maneira absoluta, o problema surge quando o “depois” não possui uma ação concreta e acaba se tornando permanente. O que não pode ocorrer é o acúmulo de dívida técnica (decisões de baixa qualidade, nomes ruins, funções grandes, códigos duplicados, soluções improvisadas etc.).
 - Código Limpo: Um código limpo é elegante e eficiente, a lógica é direta dificultando o encobrimento de bugs, as dependências são mínimas facilitando a manutenção, o tratamento de erro é completo de acordo com uma estratégia clara e o desempenho próximo do mais eficiente de a não incitar código confuso com otimizações sorrateiras. Quando se fala em elegância queremos dizer que ele deve possuir naturalidade, harmonia, leveza e simplicidade. Ele tem testes de unidade e de aceitação com nomes significativos, oferecendo apenas uma maneira de se fazer uma tarefa, e o mais importante o autor deve realmente se importar com o código.

Capítulo 2

- Problemas: Nome de variáveis, funções, classes, arquivos e diretórios que confundem o sentido do código.

- Os leitores não devem ter de traduzir mentalmente os nomes que você escolheu.
 - Linguagem informal e trocadilhos eliminam a clareza do código.
- Ideia central: A nomeação deve expressar a intenção do código, eles devem ser capazes de revelar porque algo existe dispensando o uso de comentários explicativos.
 - A nomeação deve revelar o seu propósito.
 - A nomeação não deve passar dicas/informações falsas.
 - Evitar nomes muito parecidos.
 - Usar formatos inconsistentes.
 - Usar nomes diferentes para coisas com significados diferentes.
 - Evitar redundância na nomenclatura.
 - Use nomes pronunciáveis.
 - O tamanho de um nome deve ser proporcional ao tamanho do escopo.
 - Use nomes passíveis de busca.
 - Evite prefixos.
 - Use uma palavra por conceito.
 - Use nome a partir do domínio da solução.
- Objetivo: Oferecer um conjunto de regras práticas para criar um vocabulário de código que seja claro, pronunciável e fácil de buscar, permitindo aos seus leitores uma leitura clara e de fácil interpretação.